# INHA UNIVERSITY TASHKENT
# FALL SEMESTER 2024

# SOC 3010
# OPERATING SYSTEM

## CREDITS/HOURS PER WEEK : 3/3

## COURSE TYPE : TECHNICAL CORE SEQUENCE


**INSTRUCTOR**
**DR. A. R. NASEER**
**HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG**

# SOC 3010
# OPERATING SYSTEM

## WEEK 6 LECTURE

# INTERRUPTS HANDLING

## WISH YOU ALL THE BEST

# Interrupt Handling

- Most exceptions are handled simply by sending a Unix signal to the process that caused the exception.
- The action to be taken is thus deferred until the process receives the signal; as a result, the kernel is able to process the exception quickly.
- This approach does not hold for interrupts, because they frequently arrive long after the process to which they are related, (for instance, a process that requested a data transfer) has been suspended and a completely unrelated process is running.
- So it would make no sense to send a Unix signal to the current process.

# Interrupt Handling

- **Interrupt handling depends on the type of the interrupt.**
- **There are three main classes of Interrupts:**

**I/O Interrupts**

An I/O device requires attention; the corresponding interrupt handler must query the device to determine the proper course of action.

**Timer Interrupts**

Some timer, either a local APIC timer or an external timer, has issued an interrupt; this kind of interrupt tells the kernel that a fixed-time interval has elapsed. These interrupts are mostly handled as I/O interrupts

**Interprocessor Interrupts**

A CPU issued an interrupt to another CPU of a multiprocessor system

# I/O INTERRUPT HANDLING

- Due to hardware limitations, several devices may share the same IRQ line. (Remember that PCs supply only a few IRQs.)
- This means that the interrupt vector alone does not tell the whole story: as an example, some PC configurations may assign the same vector to the network card and to the graphic card.
- Therefore, an interrupt handler must be flexible enough to service several devices.
- The interrupt handler flexibility is achieved in two distinct ways :
  - IRQ Sharing
  - IRQ Dynamic Allocation

# I/O INTERRUPT HANDLING

## IRQ Sharing

- In this scheme, the interrupt handler executes several interrupt service routines(ISRs).
- Each ISR is a function related to a single device sharing the IRQ line.
- Since it is not possible to know in advance which particular device issued the IRQ, each ISR is executed to verify whether its device needs attention; if so, the ISR performs all the operations that need to be executed when the device raises an interrupt.

# I/O INTERRUPT HANDLING

## IRQ Dynamic Allocation

- An IRQ line is associated with a device driver at the last possible moment.
- For instance, the IRQ line of the floppy device is allocated only when a user accesses the floppy disk.
- In this way, the same IRQ vector may be used by several hardware devices even if they cannot share the IRQ line; of course, the hardware devices cannot be used at the same time.

# I/O INTERRUPT HANDLING

- **Interrupt Handling**

- **Not all actions to be performed when an interrupt occurs have the same urgency.**
- **In fact, the interrupt handler itself is not a suitable place for all kind of actions.**
- **Long noncritical operations should be deferred, since while an interrupt handler is running, the signals on the corresponding IRQ line are ignored.**
- **Most important, the process on behalf of which an interrupt handler is executed must always stay in the TASK_RUNNING state, or a system freeze could occur.**
- **Therefore, interrupt handlers cannot perform any blocking procedure such as I/O disk operations.**

# I/O INTERRUPT HANDLING

- Linux divides the actions to be performed following an interrupt into three classes:

- Critical
- Noncritical
- 
- Noncritical deferrable

# I/O INTERRUPT HANDLING

- **Critical**

**Actions such as acknowledging an interrupt to the PIC, reprogramming the PIC or the device controller, or updating data structures accessed by both the device and the processor. These can be executed quickly and are critical because they must be performed as soon as possible. Critical actions are executed within the interrupt handler immediately, with maskable interrupts disabled.**

- **Noncritical**

**Actions such as updating data structures that are accessed only by the processor (for instance, reading the scan code after a keyboard key has been pushed). These actions can also finish quickly, so they are executed by the interrupt handler immediately, with the interrupts enabled.**

# I/O INTERRUPT HANDLING

- **Noncritical deferrable**
- **Actions such as copying a buffer's contents into the address space of some process (for instance, sending the keyboard line buffer to the terminal handler process).**
- **These may be delayed for a long time interval without affecting the kernel operations; the interested process will just keep waiting for the data.**
- **Noncritical deferrable actions are performed by means of separate functions called as "Softirqs and Tasklets" (earlier called as "bottom halves" )**

# I/O INTERRUPT HANDLING

**All interrupt handlers perform the same four basic actions:**

1. **Save the IRQ value and the registers contents in the Kernel Mode stack.**

2. **Send an acknowledgment to the PIC that is servicing the IRQ line, thus allowing it to issue further interrupts.**

3. **Execute the interrupt service routines (ISRs) associated with all the devices that share the IRQ.**

4. **Terminate by jumping to the ret_from_intr( ) address.**

# I/O INTERRUPT HANDLING

## Interrupt Vectors in Linux

| Vector Range | Use |
|---|---|
| 0 – 19(0x0 – 0x13) | Non-maskable Interrupts and exceptions |
| 20 – 31(0x14 – 0x1f) | Intel Reserved |
| 32 – 127(0x20 – 0x7f) | External Interrupts (IRQs) |
| 128 (0x80) | Programmed Exception for System calls |
| 129 – 238(0x81 – 0xee) | External Interrupts (IRQs) |
| 239(0xef) | Local APIC timer interrupt |
| 240(0xf0) | Local APIC thermal interrupt (introduced in the Pentium 4 models) |
| 241 – 250(0xf1 – 0xfa) | Reserved by Linux for future use |
| 251 – 253(0xfb – 0xfd) | Interprocessor Interrupts |
| 254(0xfe) | Local APIC error interrupt (generated when the local APIC detects an erroraneous condition) |
| 255(0xff) | Local APIC spurious interrupt (generated if the CPU masks an interrupt while the hardware device raises it) |

# I/O INTERRUPT HANDLING

## Interrupt Vectors

- The IBM compatible PC architecture requires that some devices must be statically connected to specific IRQ lines. In particular:
  - The interval timer device must be connected to the IRQ0
  - The slave 8259A PIC must be connected to the IRQ2 line
  - The external mathematical coprocessor must be connected to the IRQ13 line (although recent Intel 80x86 processors no longer use such a device, Linux continues to support the venerable 80386 model).
- The physical IRQs may be assigned any vector in the range 32-238.
- In general, an I/O device can be connected to a limited number of IRQ lines

# I/O INTERRUPT HANDLING

**In both cases, the kernel can retrieve the selected IRQ line of a device when initializing the corresponding driver.**

**Table shows a fairly arbitrary arrangement of devices and IRQs, such as might be found on one particular PC.**

Table . An Example of IRQ Assignment to I/O Devices

| IRQ | INT | Hardware Device |
|---|---|---|
| 0 | 32 | Timer |
| 1 | 33 | Keyboard |
| 2 | 34 | PIC cascading |
| 3 | 35 | Second serial port |
| 4 | 36 | First serial port |
| 6 | 38 | Floppy disk |
| 8 | 40 | System clock |
| 11 | 43 | Network interface |
| 12 | 44 | PS/2 mouse |
| 13 | 45 | Mathematical coprocessor |
| 14 | 46 | EIDE disk controller's first chain |
| 15 | 47 | EIDE disk controller's second chain |

# I/O INTERRUPT HANDLING

## Interrupt Vectors

- For all remaining IRQs, the kernel must establish a correspondence between IRQ number and I/O device before enabling interrupts.
- Otherwise, how could the kernel handle a signal from (say) a SCSI disk without knowing which vector corresponds to the device?
- Modern I/O devices are able to connect themselves to several IRQ lines.
- The optimal selection depends on how many devices are on the system and whether any are constrained to respond only to certain IRQs.

# I/O INTERRUPT HANDLING

## Interrupt Vectors

- **There are two ways to select a IRQ line for each device:**
  - **By a utility program executed when installing the device: such a program may ask the user to select an available IRQ number or determine an available number by itself.**
  - **By a hardware protocol executed at system startup.**

    **Under this system, peripheral devices declare which interrupt lines they are ready to use; the final values are then negotiated to reduce conflicts as much as possible.**

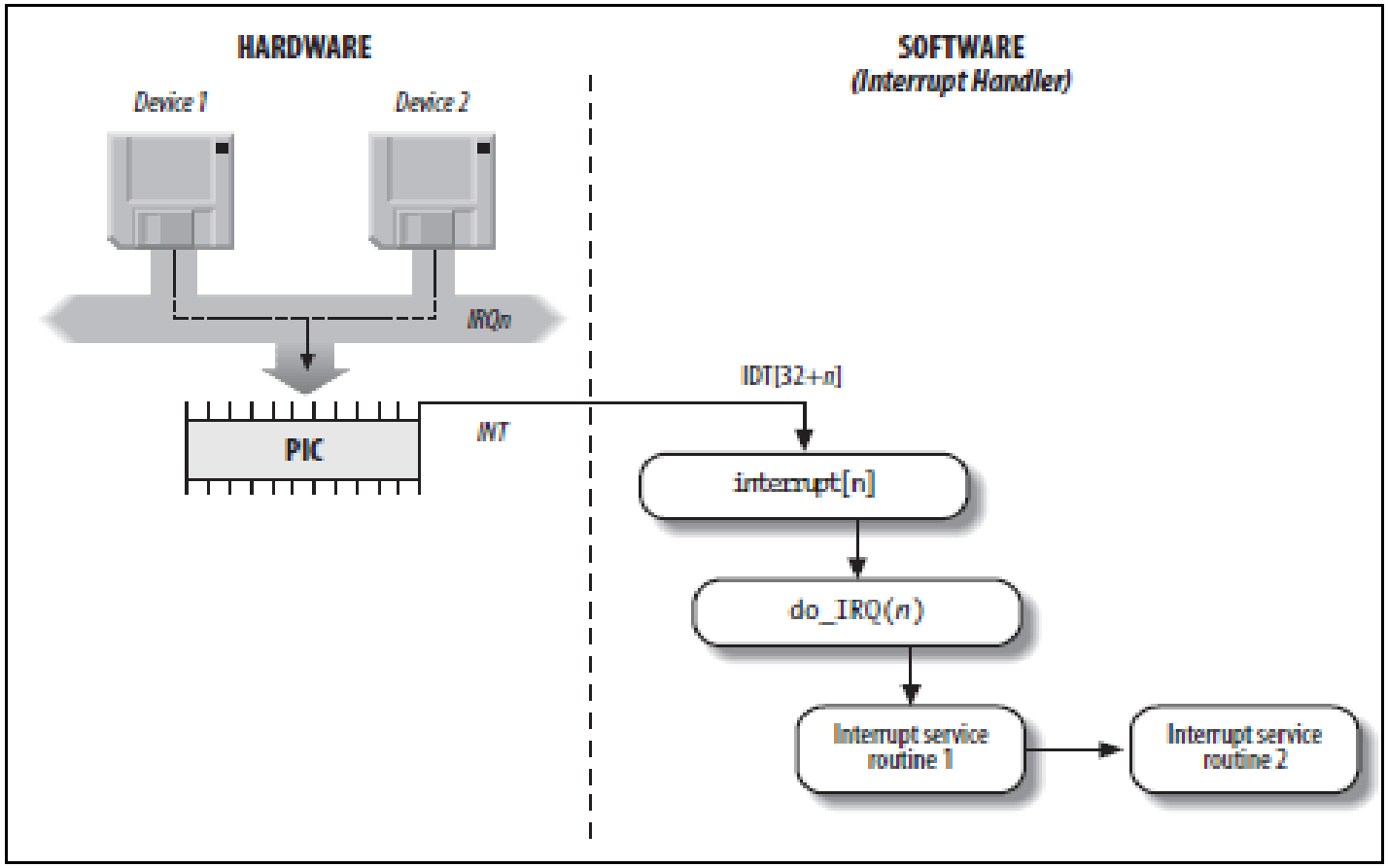
    **Once this is done, each interrupt handler can read the assigned IRQ by using a function that accesses some I/O ports of the device.**

    **For instance, drivers for devices that comply with the Peripheral Component Interconnect (PCI) standard make use of a group of functions such as pci_read_config_byte() and pci_write_config_byte() to access the device configuration space.**

# I/O INTERRUPT HANDLING

**Several descriptors are needed to represent both the state of the IRQ lines and the functions to be executed when an interrupt occurs.**

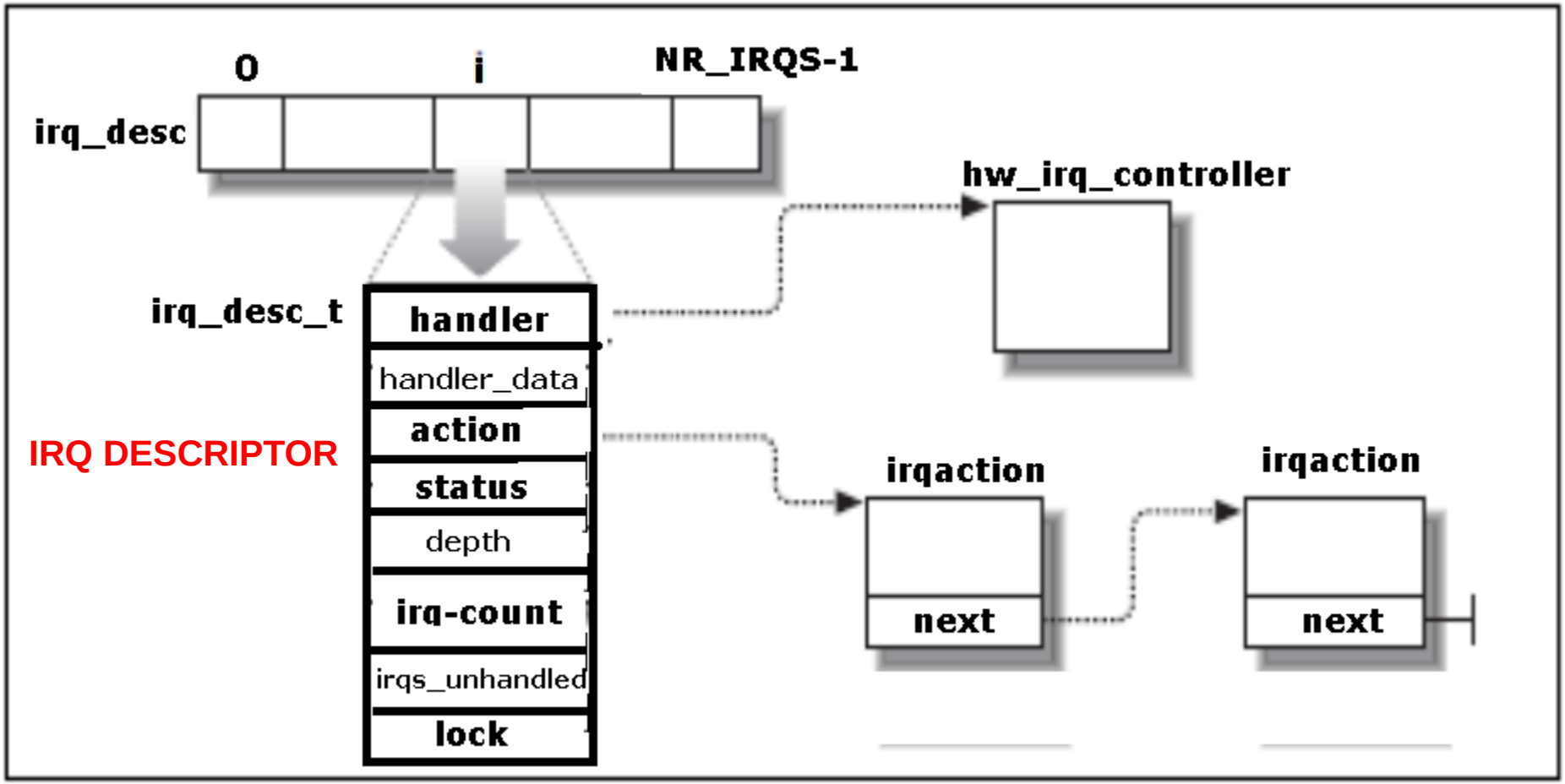
**Figure represents in a schematic way the hardware circuits and the software functions used to handle an interrupt.**



# I/O INTERRUPT HANDLING

## IRQ Data Structures that support Interrupt Handling

- Figure illustrates schematically the relationships between the main descriptors that represent the state of the IRQ lines.
- Every interrupt vector has its own irq_desc_t descriptor.
- All such descriptors are grouped together in the irq-desc array



# I/O INTERRUPT HANDLING ons

- **irq_desc_t descriptor**
  - **An irq _desc array includes NR_IRQS irq _desc_t descriptors, which include the following fields:**

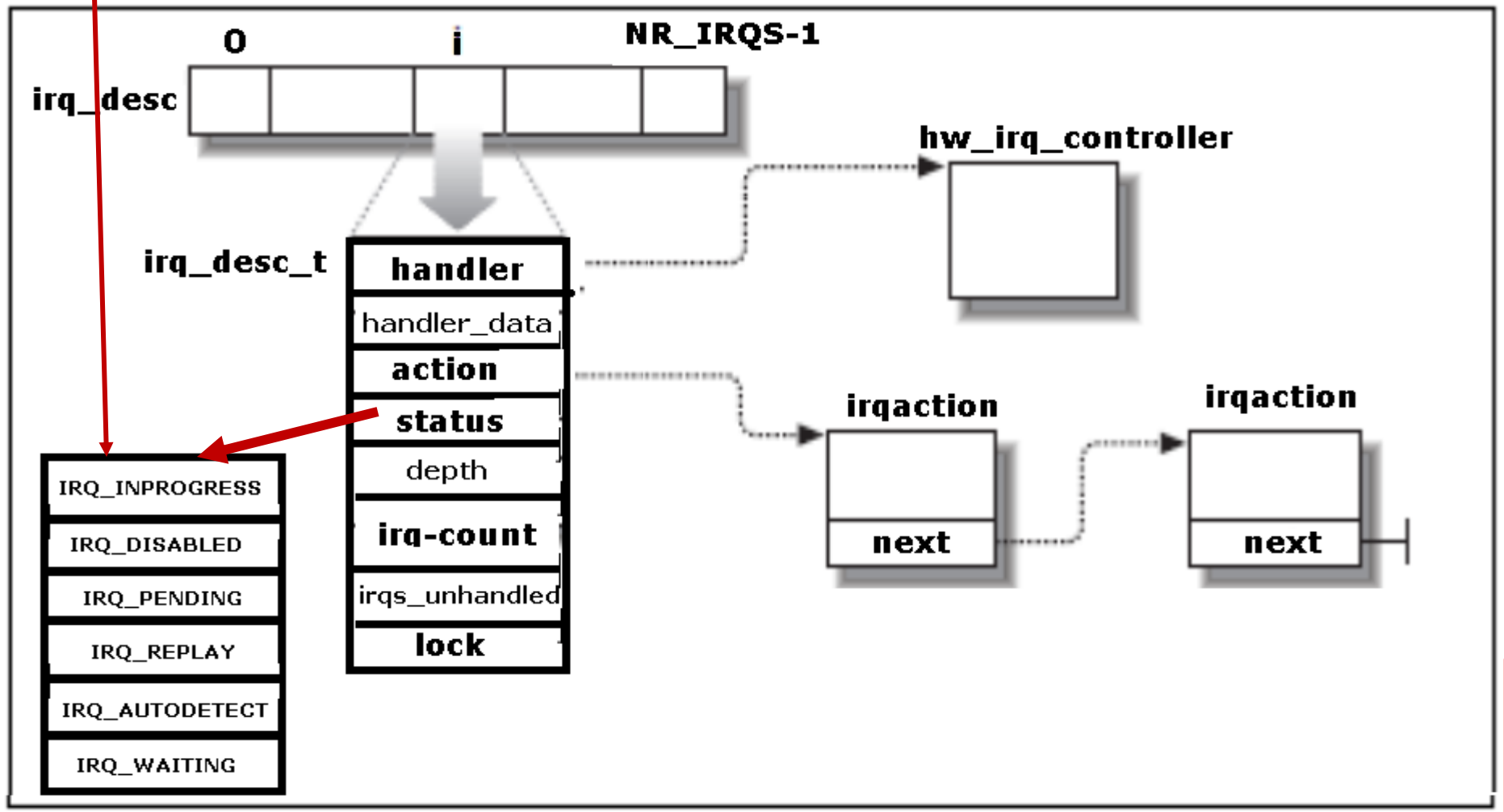
| Field | Description |
|---|---|
| handler | Points to the hw_interrupt_type descriptor that identifies the PIC circuit servicing the IRQ line. |
| handler_data | Pointer to data used by the PIC methods |
| action | Identifies the interrupt service routines to be invoked when the IRQ occurs. The field points to the first element of the list of descriptors associated with the IRQ. |
| status | A set of flags describing the IRQ line status. |
| depth | Shows 0 if the IRQ line is enabled and a positive value if it has been disabled at least once. |
| irq_count | Counter of interrupt occurrences on the IRQ line (for diagnostic use only) |
| irqs_unhandled | Counter of unhandled interrupt occurrences on the IRQ line (for diagnostic use only) |
| lock | A spin lock used to serialize the accesses to the IRQ descriptor and to the PIC |

# I/O INTERRUPT HANDLING

- irq_desc_t descriptor

**status - A set of flags describing the IRQ line status.**

**IRQ line status flags**



# I/O INTERRUPT HANDLING

- **irq_desc_t descriptor**

**status - A set of flags describing the IRQ line status.**

**IRQ line status flags which include the following fields:**

| Flag name | Description |
|---|---|
| IRQ _INPROGRESS | A handler for the IRQ is being executed. |
| IRQ _DISABLED | The IRQ line has been deliberately disabled by a device driver. |
| IRQ _PENDING | An IRQ has occurred on the line; its occurrence has been acknowledged to the PIC, but it has not yet been serviced by the kernel. |
| IRQ _REPLAY | The IRQ line has been disabled but the previous IRQ occurrence has not yet been acknowledged to the PIC. |
| IRQ _AUTODETECT | The kernel is using the IRQ line while performing a hardware device probe. |
| IRQ _WAITING | The kernel is using the IRQ line while performing a hardware device probe; moreover, the corresponding interrupt has not been raised. |

# I/O INTERRUPT HANDLING

- **irq_desc_t descriptor - unexpected interrupts**
  - **An interrupt is unexpected if it is not handled by the kernel, that is, either if there is no ISR associated with the IRQ line, or if no ISR associated with the line recognizes the interrupt as raised by its own hardware device.**
  - **Usually the kernel checks the number of unexpected interrupts received on an IRQ line, so as to disable the line in case a faulty hardware device keeps raising an interrupt over and over.**
  - **Because the IRQ line can be shared among several devices, the kernel does not disable the line as soon as it detects a single unhandled interrupt.**
  - **Instead, the kernel stores in the irq_count and irqs_unhandled fields of the irq-desc_t descriptor the total number of interrupts and the unexpected interrupts, respectively,; when the 100,000$^{th}$ interrupt is raised, the kernel disables the line if the number of unhandled interrupts is above 99,900(that is, if less than 101 interrupts over the last 100,000 received are expected interrupts from hardware devices sharing the line).**

# I/O INTERRUPT HANDLING

- **irq_desc_t descriptor – IRQ line enable/disable**

- **The depth field and the IRQ_DISABLED flag of the irq_desc_t descriptor specify whether the IRQ line is enabled or disabled.**

- **Every time the disable_irq( ) function is invoked, it increments depth field;**
- **if depth is equal to 0, the function disables the IRQ line and set its IRQ_DISABLED flag**

- **Conversely, each invocation of the enable_irq( ) function decrements the field; if depth becomes 0, the function enables the IRQ line and clears its IRQ_DISABLED flag**

# I/O INTERRUPT HANDLING

## hw_interrupt_type descriptor

- Linux supports, in addition to the 8259A chip , several other PIC circuits such as the SMP IO-APIC, PIIX4's internal 8259 PIC, and SGI's Visual Workstation Cobalt (IO-)APIC.
- To handle all such devices in a uniform way, Linux uses a PIC object, consisting of the PIC name and 7 PIC standard methods
- The data structure that defines a PIC object is called hw_interrupt_type (also called hw_irq_controller)
- This descriptor includes a group of pointers to the low-level I/O routines that interact with a specific PIC circuit.

# I/O INTERRUPT HANDLING

## The hw_interrupt_type descriptor

- Consider computer is a uniprocessor with two 8259A PICs, which provides the 16 standard IRQ.
- In this case, the handler field in each of the irq _desc_t descriptors points to the i8259A_irq _type variable, which describes the 8259A PIC.
- This variable is initialized as follows:

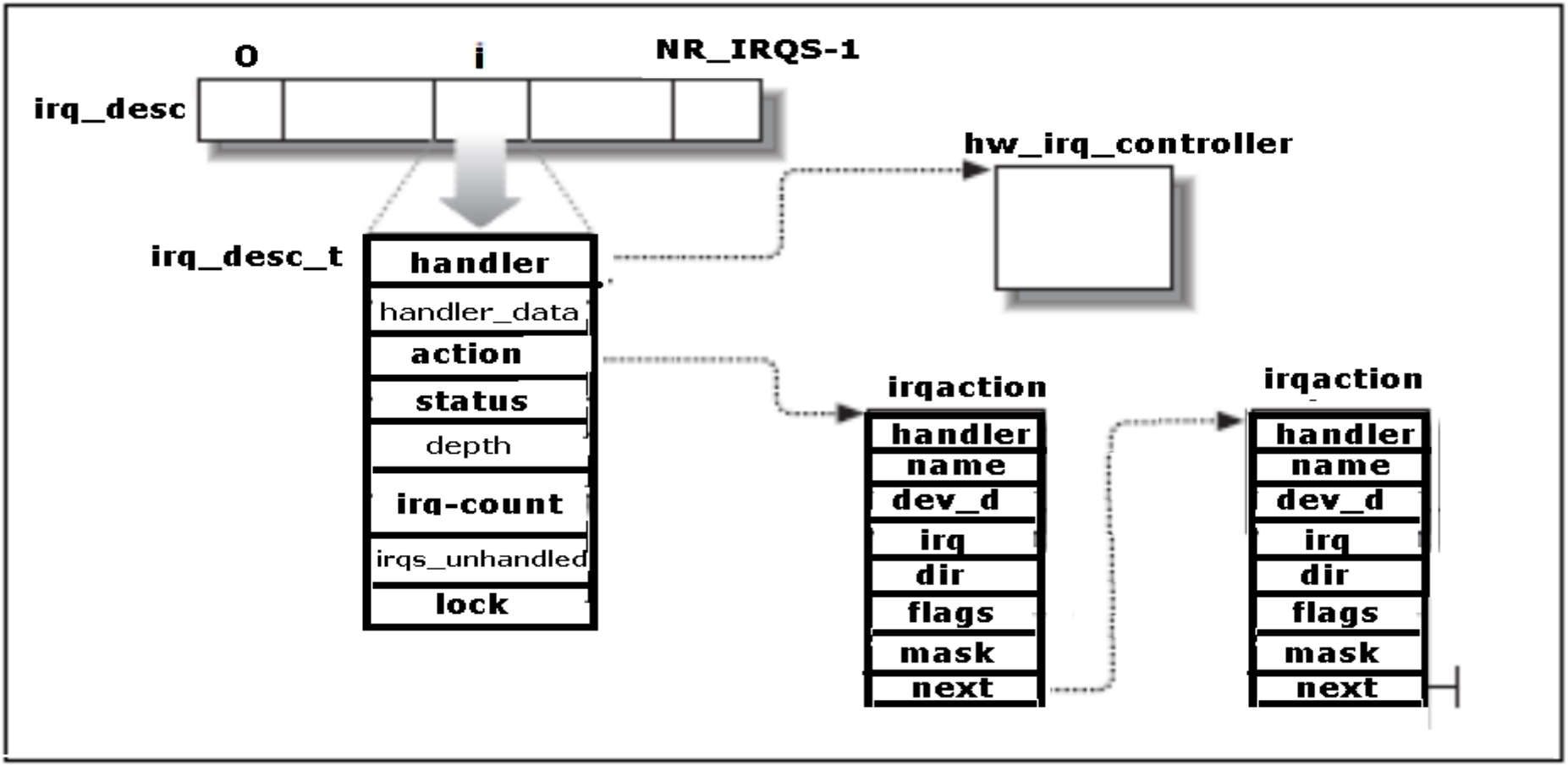
```
struct hw_interrupt_type i8259A_irq_type = {
    .typename       = "XT-PIC",          # The first field in this structure, "XT-PIC", is a name.
    .startup        = startup_8259A_irq,
    .shutdown       = shutdown_8259A_irq,
    .enable         = enable_8259A_irq,
    .disable        = disable_8259A_irq
    .ack            = mask_and_ack_8259A;
    .end            = end_8259A_irq;
    .set_affinity   = NULL };
```

Following i8259A_irq_type includes pointers to five different functions used to program the PIC. The first two functions start up and shut down an IRQ line of the chip, respectively. But in the case of the 8259A chip these functions coincide with the last two functions, which enable and disable the line. The mask_and_ack_8259() function acknowledges the IRQ received by sending the proper bytes to the 8259A I/O ports. The end_8259A_irq function is invoked when the interrupt handler for the IRQ line terminates. The last set_affinity method is set to NULL – it is used in multiprocessor systems to declare the affinity of CPUs for specified IRQs - that is, which CPUs are enabled to handle specific IRQs

# I/O INTERRUPT HANDLING

## The irqaction descriptor

- Multiple devices can share a single IRQ.
- Therefore, the kernel maintains irqaction descriptors, each of which refers to a specific hardware device and a specific interrupt. The descriptor includes the following fields.



# I/O INTERRUPT HANDLING

## The irqaction descriptor

- Multiple devices can share a single IRQ.
- Therefore, the kernel maintains irqaction descriptors, each of which refers to a specific hardware device and a specific interrupt. The descriptor includes the following fields.

| Field Name | Description |
|---|---|
| handler | Points to the interrupt service routine for an I/O device. This is the key field that allows many devices to share the same IRQ. |
| name | Names of the I/O device (shown when listing the serviced IRQs by reading the /proc/interrupts file). |
| dev_id | A private field for the I/O device. Typically, it identifies the I/O device itself |
| next | Points to the next element of a list of irqaction descriptors. The elements in the list refer to hardware devices that share the same IRQ. |
| irq | IRQ line |
| dir | Points to the descriptor of the /proc/irq/n directory associated with the IRQn |
| flags | Describes the relationships between IRQ line and I/O device in a set of flags |
| Mask | Not used |

# I/O INTERRUPT HANDLING

- **Flags of the irqaction descriptor**

| | |
|---|---|
| **IRQF_INTERRUPT** | **The handler must execute with interrupts disabled.** |
| **IRQF_SHIRQ** | **The device permits its IRQ line to be shared with other devices.** |
| **IRQF_SAMPLE_RANDOM** | **The device may be considered as a source of events occurring randomly; it can thus be used by the kernel random number generator. (Users can access this feature by taking random numbers from the /dev/random and /dev/urandom device files.)** |

# I/O INTERRUPT HANDLING

- **The irqaction descriptor**

**The irq_stat array includes NR_CPUS entries, one for every possible CPU in the system.**

**Each entry of type irq_cpustat_t includes a few counters and flags used by the kernel to keep track of what each CPU is currently doing.**

**Fields of the irq_cpustat_t structure**

| | |
|---|---|
| __softirq_pending | Set of flags denoting the pending sofirqs |
| Idle_timestamp | Time when the CPU became idle (significant only if the CPU is currently idle) |
| __nmi_count | Number of occurrences of NMI interrupts |
| apic_timer_irqs | Number of occurrences of local APIC timer interrupts |

# Hardware Handling of Interrupts and Exceptions

## IRQ Distribution in Multiprocessor Systems

- Linux sticks to the Symmetric Multiprocessing Model (SMP); that means, essentially, that the kernel should not have any bias toward one CPU with respect to the others
- As a consequence, the kernel tries to distribute the IRQ signals coming from the hardware devices in a round-robbin fashion among all the CPUs.
- Therefore, all the CPUs should spend approximately the same fraction of their execution time servicing I/O interrupts
- During the system bootstrap, the booting CPU executes the setup_IO_APIC_irqs() function to initialize the I/O APIC chip.
- The 24 entries of the Interrupt Redirection Table of the chip are filled, so that all IRQ signals from the I/O hardware devices can be routed to each CPU in the system according to the “lowest priority” scheme.

# I/O INTERRUPT HANDLING

## IRQ Distribution in Multiprocessor Systems

- During System boostrap, all CPUs execute the setup_local_APIC() function, which takes care of initializing the local APICs.
- In particular, the task priority register (TPR) of each chip is initialized to a fixed value, meaning that the CPU is willing to handle every kind of IRQ signal, regardless of its priority.
- The Linux kernel never modifies this value after its initialization
- All task priority registers contain the same value, thus all CPUs always have the same priority.
- To break a tie, the multi-APIC system uses the values in the arbitration registers of local APICs.
- Because such values are automatically changed after every interrupt, the IRQ signals are, in most cases, fairly distributed among all CPUs.

# I/O INTERRUPT HANDLING

## IRQ Distribution in Multiprocessor Systems

- In short, when a hardware device raises an IRQ signal, the multi-APIC system selects one of the CPUs and delivers the signal to the corresponding local APIC, which in turn, interrupts its CPU.
- No other CPUs are notified of the event.
- All this magically done by the hardware, so it should be of no concern for the kernel after the multi-APIC system initialization.
- If the hardware fails to distribute the interrupts among the CPUs in a fair way, Linux makes use of a special kernel thread called **kirqd** to correct, if necessary, the automatic assignment of IRQs to CPUs
- The kernel thread exploits a nice feature of multi-APIC systems, called **the IRQ affinity of a CPU,** by modifying the Interrupt Redirection Table entries of the I/O APIC, it is possible to route an interrupt signal to a specific CPU.
- This can be done by the invoking the **set_ioapic_affinity_irq()** function, which acts on two parameters: the IRQ vector to be routed and a 32-bit mask denoting the CPUs that can receive the IRQ.

# Hardware Handling of Interrupts and Exceptions

## IRQ Distribution in Multiprocessor Systems

- The IRQ affinity of a given interrupt also can be changed by the system administrator by writing a new CPU bitmap mask into the /proc/irq/n/smp_affinity file (where n being the interrupt vector)
- The kirqd kernel thread periodically executes the do_irq_balance() function, which keeps track of the number of interrupt occurences received by every CPU in the most recent time interval.
- If the function discovers that the IRQ load imbalance between the heaviest loaded CPU and the least loaded CPU is significantly high, then it either selects an IRQ to be moved from a CPU to another, or rotates all IRQs among all existing CPUs.

# MULTIPLE KERNEL MODE STACKS

**Kernel makes use of three types of Kernel Mode Stacks :**

- **Exception Stack**
  - **Used when handling exceptions (including system calls)**
  - **Kernel makes use of a different exception stack for each process in the system**
- **Hard IRQ stack**
  - **Used when handling interrupts**
  - **There is one Hard IRQ stack for each CPU in the system and each stack is contained in a single page frame.**
- **Soft IRQ Stack**
  - **Used when handling deferrable functions**
  - **There is one Soft IRQ stack for each CPU in the system and each stack is contained in a single page frame.**

# Hardware Handling of Interrupts and Exceptions

## The do_IRQ( ) Function

- **The do_IRQ( ) function is invoked to execute all interrupt service routines associated with an interrupt.**

- **When it starts, the kernel stack contains from the top down:**
  - **The do_IRQ( ) return address**
  - **The group of register values pushed on by SAVE_ALL**
  - **The encoding of the IRQ number**
  - **The registers saved automatically by the control unit when it recognized the interrupt**

## The do_IRQ( ) Function

- Since the C compiler places all the parameters on top of the stack, the do_IRQ( ) function is declared as follows:

```
void do_IRQ(struct pt_regs regs)
```

where the pt_regs structure consists of 15 fields:

- The first nine fields correspond to the register values pushed by SAVE_ALL.
- The tenth field, referenced through a field called orig_eax, encodes the IRQ number.
- The remaining fields correspond to the register values pushed on automatically by the control unit.

# Hardware Handling of Interrupts and Exceptions

## Interrupt Service Routines

- An interrupt service routine implements a device-specific operation.
- All of them act on the same parameters:

**irq**

The IRQ number

**dev_id**

The device identifier

**regs**

A pointer to the Kernel Mode stack area containing the registers saved right after the interrupt occurred

- The first parameter allows a single ISR to handle several IRQ lines,
- the second one allows a single ISR to take care of several devices of the same type, and
- the last one allows the ISR to access the execution context of the interrupted kernel control path.

# Hardware Handling of Interrupts and Exceptions

## Dynamic Handling of IRQ Lines

- With the exception of IRQ0, IRQ2, and IRQ13, the remaining 13 IRQs are dynamically handled.
- There is, therefore, a way in which the same interrupt can be used by several hardware devices even if they do not allow IRQ sharing: the trick consists in serializing the activation of the hardware devices so that just one at a time owns the IRQ line.
- Before activating a device that is going to make use of an IRQ line, the corresponding driver invokes request_irq( ).
- This function creates a new irqaction descriptor and initializes it with the parameter values; it then invokes the setup_x86_irq( ) function to insert the descriptor in the proper IRQ list.
- The device driver aborts the operation if setup_x86_irq( ) returns an error code, which means that the IRQ line is already in use by another device that does not allow interrupt sharing.
- When the device operation is concluded, the driver invokes the free_irq( ) function to remove the descriptor from the IRQ list and release the memory area.

# Hardware Handling of Interrupts and Exceptions

## Dynamic Handling of IRQ Lines

- Let us see how this scheme works on a simple example.
- Assume a program wants to address the /dev/fd0 device file, that is, the device file that corresponds to the first floppy disk on the system.
- The program can do this either by directly accessing /dev/fd0 or by mounting a filesystem on it.
- Floppy disk controllers are usually assigned IRQ6; given this, the floppy driver will issue the following request:

```
request_irq(6, floppy_interrupt,
IRQF_INTERRUPT|IRQF_SAMPLE_RANDOM, "floppy", NULL);
```

- As can be observed, the floppy_interrupt( ) interrupt service routine must execute with the interrupts disabled (IRQF_INTERRUPT set) and no sharing of the IRQ (IRQF_SHIRQ flag cleared).
- When the operation on the floppy disk is concluded (either the I/O operation on /dev/fd0 terminates or the filesystem is unmounted), the driver releases IRQ6:

```
free_irq(6, NULL);
```

# Hardware Handling of Interrupts and Exceptions

## Dynamic Handling of IRQ Lines

- In order to insert an irqaction descriptor in the proper list, the kernel invokes the setup_x86_irq( ) function, passing to it the parameters irq _nr, the IRQ number, and new, the address of a previously allocated irqaction descriptor.

- This function:
  - Checks whether another device is already using the irq _nr IRQ and, if so, whether the IRQF_SHIRQ flags in the irqaction descriptors of both devices specify that the IRQ line can be shared. Returns an error code if the IRQ line cannot be used.
  - Adds *new (the new irqaction descriptor) at the end of the list to which irq _desc[irq _nr]->action points.
  - If no other device is sharing the same IRQ, clears the IRQ _DISABLED and IRQ _INPROGRESS flags in the flags field of *new and reprograms the PIC to make sure that IRQ signals are enabled.

# Interprocessor Interrupt Handling

- Inter-processor interrupts allow a CPU to send interrupt signals to any other CPU in the system.
- An inter-processor interrupt (IPI) is delivered not through an IRQ line, but directly as a message on the bus that connects the local APIC of all CPUs (either a dedicated bus in older motherboards, or the system bus in the Pentium 4-based motherboards, or the system bus in the Pentium 4-based motherboards).
- On multiprocessor systems, Linux makes use of three kinds of inter-processor interrupts

  CALL_FUNCTION_VECTOR (vector 0xfb)

  RESHEDULE_VECTOR (vector 0xfc)

  INVALIDATE_TLB_VECTOR (vector 0xfd)

# Interprocessor Interrupt Handling

**On multiprocessor systems, Linux makes use of three kinds of inte-rprocessor interrupts**

- **CALL_FUNCTION_VECTOR (vector 0xfb)**

  - **Sent to all CPUs but the sender, forcing those CPUs to run a function passed by the sender. The corresponding interrupt handler is named call_function_interrupt( ).**
  - **The function (whose address is passed in the call_data global variable) may, for instance, force all other CPUs to stop, or may force them to set the contents of the Memory type range Registers (MTRRs).**
  - **Usually this interrupt is sent to all CPUS except the CPU executing the calling function by means of the smp_call_function( ) facility function.**

- **RESHEDULE_VECTOR (vector 0xfc)**

  - **When a CPU receives this type of interrupt, the corresponding handler-named reschedule_interrupt ( )-limits itself to acknowledging the interrupt.**
  - **Rescheduling is done automatically when returning from the interrupt**

- **INVALIDATE_TLB_VECTOR (vector 0xfd)**

  - **Sent to all CPUs but the sender, forcing them to invalidate their Translation Lookaside Buffers. The corresponding handler, named invalidate_interrupt ( ) flushes some TLB entries of the processor**

# Softirqs and Tasklets

- In "Interrupt Handling" that several tasks among those executed by the kernel are not critical: they can be deferred for a long period of time, if necessary.
- Remember that the interrupt service routines of an interrupt handler are serialized, and often there should be no occurrence of an interrupt until the corresponding interrupt handler has terminated.
- Conversely, the deferrable tasks can execute with all interrupts enabled.
- Taking them out of the interrupt handler helps keep kernel response time small.
- This is a very important property for many time critical applications that expect their interrupt requests to be serviced in a few milliseconds.
- Linux answers such a challenge by using two kinds of non urgent interruptible kernel functions: the so called deferrable functions' (softriqs and tasklets), and those executed by means of some work queues

# Softirqs And Tasklets

## Softirqs

- Softriqs are statically allocated (i.e., defined at compile time), while tasklets can be allocated and initialized at runtime also (for instance, when loading a kernel module).
- Softriqs can run concurrently on several CPUs, even if they are of the same type.
- Thus, softriqs are reentrant functions and must explicitly protect their data structures with spin locks.

## Tasklets

- Tasklets do not have to worry about this, because their exception is controlled more strictly by the kernel.
- Tasklets of the same type are always serialized: in other words, the same type of tasklet cannot be executed by two CPUs at the same time.
- However, tasklets of different types can be executed concurrently on several CPUs.
- Serializing the tasklet simplifies the life of device driver developers because the tasklet function needs not to be reentrant.
- Softirqs and tasklets are strictly correlated, because tasklets are implemented on top of softirqs.

# Softirqs And Tasklets

## Softirqs And Tasklets

**Generally speaking, four kinds of operation can be performed on deferrable functions:**

### Initialization

**Defines a new deferrable function; this operation is usually done when the kernel initializes itself or a module is loaded.**

### Activation

**Marks a deferrable function as "pending" - to be run the next time the kernel schedules a round of executions of deferrable functions.**

**Activation can be done at any time (even while handling interrupts).**

### Masking

**Selectively disables a deferrable function so that will not be executed by the kernel even if activated. disabling deferrable functions is sometimes essential.**

### Execution

**Executes a pending deferrable function together with all other pending deferrable functions of the same type; execution is performed at well-specified times,**

# Softirqs

## Softriqs

- Linux uses a limited number of softirqs.
- For most purposes, tasklets are good enough and are much easier to write because they do not need to be re-entrant.
- As a matter of fact, only the six kinds of softriqs listed in Table are currently defined.

## Table Softirqs used in Linux

| Softriq | Index (Priority) | Description |
|---|---|---|
| HI_SOFTIRQ | 0 | Handles high priority tasklets |
| Timer_SOFTIRQ | 1 | tasklets related to timer interrupts |
| NET_TX_SOFTIRQ | 2 | Transmits packets to network cards |
| NET_RX_SOFTIRQ | 3 | Receives packets from network cards |
| SCSI_SOFTIRQ | 4 | Post-interrupt processing of SCSI commands |
| TASKLET_SOFTIRQ | 5 | Handles regular tasklets |

The index of a softirq determines its priority, a lower index means higher priority because softirq functions will be executed starting from index 0

## TASKLETS

- Tasklets are the preferred way to implement defferable functions in I/O drivers.
- Tasklets are built on top of two softirqs named HI_SOFTIRQ and TASKLET_SOFTIRQ.
- Several tasklets may be associated with the same softirq, each tasklet carrying its own function.
- There is no real diffference between the two softirqs, except that do_softirq() executes HI_SOFTIRQ's tasklets before TASKLET_SOFTIRQ's tasklets.
- Tasklets and high priority tasklets are stored in the tasklet_vec and tasklet_hi_vec arrays, respectively.
- Both of them include NR_CPUS elements of type tasklet_head, and each element consists of a pointer to a list of tasklet descriptors.

## TASKLETS

- The tasklet descriptor is a data structure of type tasklet_struct, whose fields are shown in Table

**The fields of the tasklet descriptor**

| Field name | Description |
|---|---|
| next | pointer to next descriptor in the list |
| state | Status of the tasklet |
| count | Lock counter |
| funct | Pointer to the tasklet function |
| data | An unsigned long integer that may be used by the tasklet function |

## WORK QUEUES

- The work queues allow kernel functions to be activated (much like deferrable functions) and later executed by special kernel threads called worker threads.
- Despite their similarities, deferrable functions and work queues are quite different.
- The main difference is that deferrable functions run in interrupt context while functions in work queues run in process context.
- Running in process context is the only way to execute functions that can block (for instance, functions that need to access some block of data on disk), no process switch can take place in interrupt context.
- Neither deferrable functions nor functions in a work queue can access the User Mode address space of a process.
- In fact, a deferrable function cannot make any assumption about the process that is currently running when it is executed.
- On the other hand, a function in a work queue is executed by a kernel thread, so there is no User Mode address space to access.

# SOC 3010 OPERATING SYSTEMS

## Reading Assignments

## Chapter 4 of Text Book

## Text Book

- **Understanding the Linux Kernel**
  - **Daniel P/ Bovet and Marco Cesati**
    **Edition, O' Reilly Media 2005, ISBN 978-0596005658**

# INTERRUPTS & EXCEPTIONS

## Questions

- What are the three main classes of interrupts that are to be handled based on the interrupting devices that generate the interrupts?
- Explain the following :
  i) I/O interrupts ii) Timer interrupts iii) Inter-processor interrupts
- What are the two distinct ways by which Interrupt handler flexibility is achieved?
- Explain the following :
  i) IRQ sharing ii) IRQ dynamic allocation
- What are the two ways used to select an IRQ line for each I/O device?
- If two or more CPUs share the lowest priority, the load is distributed between them using a technique called ……..
- Describe the arbitration technique used to distribute the interrupt requests to different cores of same priority in a round-robin fashion.
- What are the three classes into which the actions to be performed following an interrupt are classified?
- Noncritical deferrable actions are performed by means of separate functions called as ……. and ……
- Explain the following actions to be performed following an interrupt
  i) Critical ii) Noncritical iii) Noncritical deferrable
- What are the four basic actions that are performed by all interrupt handlers?
- Interrupt vectors normally associated with physical IRQs - .....
  a) 0-19 b) 20-31 c)32-127 d) 251-253
- Interrupt vectors normally associated with Non-maskable interrupts and exceptions - .....
  a) 0-19 b) 20-31 c)32-127 d) 251-253
- Interrupt vectors normally associated with Inter-processor interrupts - ……..
  a) 0-19 b) 20-31 c) 32-127 d) 251-253
- IRQ number ..... and int vector ..... associated with the hardware device Timer
  a) 10, 42 b) 13, 45 c) 0, 32 d) 1, 33
- IRQ number ..... and int vector ..... associated with the hardware device Keyboard
  a) 10, 42 b) 13, 45 c) 0, 32 d) 1, 33

# INTERRUPTS & EXCEPTIONS

## Questions

- The int vector associated with the NMI is .....
- The int vector associated with the exception "Divide Error" is .....
- IThe int vector associated with the exception "Breakpoint " is .....
- The int vector associated with the exception "Page fault" is .....
- The int vector associated with the exception " Bounds check" is .....
- The int vector associated with the exception "Overflow" is …....
- The int vector associated with the exception "Double fault" is .....
- The int vector associated with the System Call is …....
- Explain what is meant by “Double fault”.
- Critical actions are executed within the interrupt handler immediately, with maskable interrupts ……
- Non-Critical actions are executed by the interrupt handler immediately, with the interrupts ……
- Write the interrupt vectors associated with the following :
- Non-maskable Interrupts and exceptions : ……
- External Interrupts (IRQs) : ……
- Programmed Exception for System calls : ……
- Local APIC timer interrupt : ……
- Local APIC thermal interrupt : ……
- Interprocessor Interrupts : ……
- Local APIC error interrupt : ……
- Local APIC spurious interrupt : ……
- Describe the irq_desc_t descriptor structure with the help of a diagram and indicate the various fields in the descriptor.
- What is meant by unexpected interrupts? How are unexpected interrupts handled in Linux?

# INTERRUPTS & EXCEPTIONS

## Questions

- State the purpose of the following fields in the irq_desc_t descriptor structure:
  a) irq_count b) irqs-unhandled
- Describe the hw_interrupt_type descriptor.
- Describe the irqaction descriptor specifying their fields.
- List the three flags of the irqaction descriptor and indicate their purpose.
- State the purpose of the following flags in irqaction descriptor:
- IRQF_INTERRUPT (SA__INTERRUPT)
- IRQF_SHARED (SA_SHIRQ)
- IRQF_SAMPLE_RANDOM (IRQF_ SAMPLE_RANDOM)
- Describe the fields in each entry of the irq_cpustat_t structure.
- Explain IRQ distribution in Multiprocessor Systems
- State the purpose of special kernel thread kirqd
- Describe IRQ affinity of a CPU in a multi-APIC system
- The kernel thread kirqd exploits a nice feature of multi-APIC systems, called the ……., by modifying the Interrupt Redirection Table entries of the I/O APIC, it is possible to route an interrupt signal to a specific CPU.
- The IRQ affinity of a given interrupt also can be changed by the system administrator by writing a new CPU bitmap mask into the ……. File.
- Describe the task performed by the do_irq_balance() function.
- What are the three types of Kernel Mode stacks?
- …….. stack is used when handling exceptions (including system calls)
- …… stack is used when handling interrupts (critical and Non-critical interrupt functions)
- …… stack is used when handling deferrable functions
- Kernel makes use of ……. exception stack for each process in the system

# INTERRUPTS & EXCEPTIONS

## Questions

- Kernel makes use of ……. Hard IRQ stack for each CPU in the system and each stack is contained in a …… page frame.
- Kernel makes use of ……. Soft IRQ stack for each CPU in the system and each stack is contained in a …… page frame.
- Explain the following :
  i) Kernel Exception stack ii) Kernel Hard IRQ stack iii) Kernel Soft IRQ stack
- Describe the do_IRQ function.
- Describe the operations performed by the following functions. Also state the purpose of all the parameters used in these functions
  request_irq(6, floppy_interrupt, IRQF_INTERRUPT|IRQF_SAMPLE_RANDOM, "floppy", NULL);
  free_irq(6, NULL);
- Describe the three kinds of inter-processor interrupts used by linux on multiprocessor systems. Also indicate the interrupt vectors associated with these.
- Explain the following inter-processor interrupts :
  CALL_FUNCTION_VECTOR()
  RESCHEDULE_VECTOR()
  INVALIDATE_TLB_VECTOR
- Distinguish between Softirqs and Tasklets.
- What are deferrable functions? Describe the four kinds of operations that can be performed on deferrable functions.
- List the six kinds of Softirqs used in Linux.
- Tasklets are built on top of two softirqs named …… and ……
- Describe the fields in the tasklets descriptor.
- Describe the tasks performed by the following functions :
  a) request_irq(() b) free_irq c) setup_x86_irq()
- Explain Work Queues.

# SOC 3010
# OPERATING SYSTEM

## END OF
## WEEK 6 LECTURE

# INTERRUPTS HANDLING

## WISH YOU ALL THE BEST

# INHA UNIVERSITY TASHKENT
# FALL SEMESTER 2024

# SOC 3010
# OPERATING SYSTEM

## CREDITS/HOURS PER WEEK : 3/3

## COURSE TYPE : TECHNICAL CORE SEQUENCE


**INSTRUCTOR**
**DR. A. R. NASEER**
**HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG**